

Optimization Techniques: Exercise 2

Maria Charisi 10727

December 2025

Github Repo

Abstract

This report constitutes the third part of the assignment for the *Optimization Techniques* course at the *Aristotle University of Thessaloniki*. The purpose of this work is to investigate the performance of two gradient-based optimization methods: the classical Steepest Descent algorithm with a fixed step size, and the Steepest Descent method with projection onto a closed convex set. We apply both methods to a quadratic objective function and analyze their behavior under various choices of the initial point and step parameter, highlighting the resulting differences in convergence.

Contents

1	Introduction	2
2	Steepest Descent Method	3
2.1	Method Description	3
2.2	Convergence Analysis for $f(x, y)$	3
2.3	Results	4
3	Projected Steepest Descent Method	5
3.1	Method Description	5
3.2	Results	7
3.2.1	Case 1: $x_0 = (5, -5)$, $\gamma = 0.5$, $s = 5$	7
3.2.2	Case 2: $x_0 = (-5, 10)$, $\gamma = 0.1$, $s = 15$	8
3.2.3	Case 3: $x_0 = (8, -10)$, $\gamma = 0.2$, $s = 0.1$	10

1 Introduction

In this assignment, we study the behavior of the gradient-based optimization method, Steepest Descent, when applied to both unconstrained and constrained problems. The objective function under consideration is the quadratic function

$$f(x, y) = \frac{1}{3}x^2 + 3y^2,$$

which is strictly convex, continuously differentiable, and has a unique global minimum at

$$(x^*, y^*) = (0, 0), \quad f(x^*, y^*) = 0.$$

Figures 1 and 2 illustrate the surface of the function in 3-dimensional space and its contour curves in the plane, respectively.

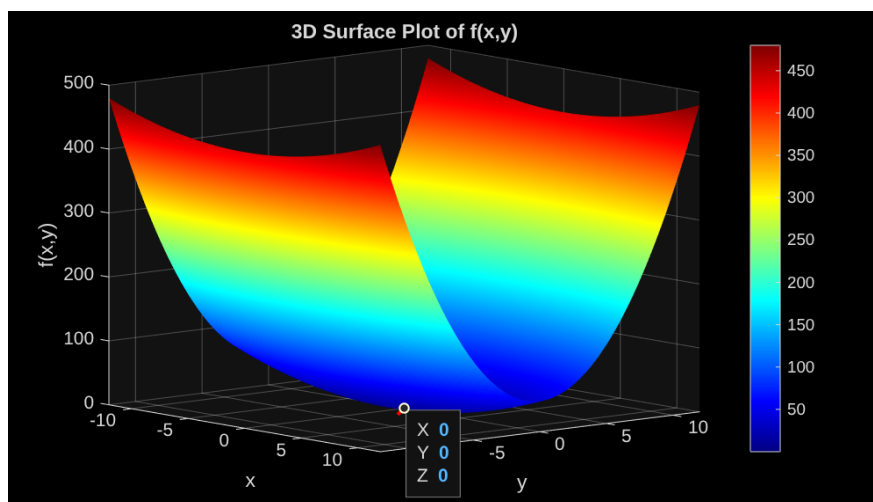


Figure 1: 3D surface plot of $f(x, y)$.

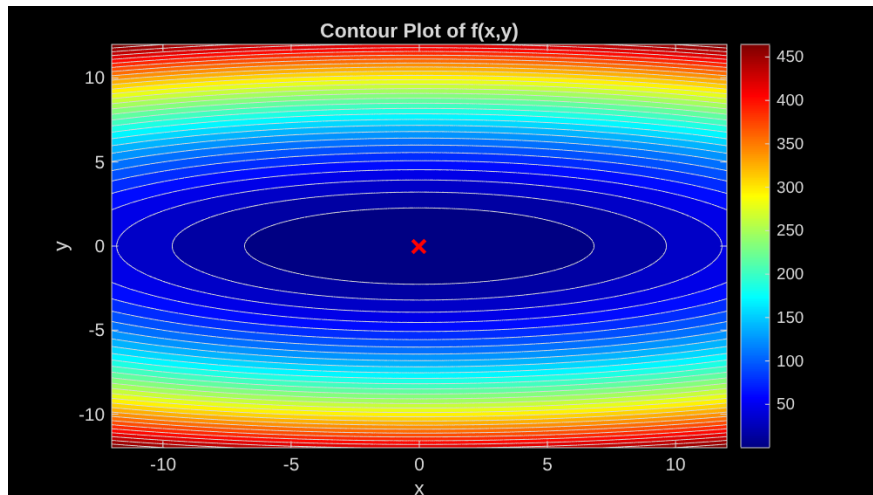


Figure 2: Contour plot of $f(x, y)$.

2 Steepest Descent Method

2.1 Method Description

The Steepest Descent method is one of the simplest iterative optimization algorithms. At each iteration, the descent direction is chosen as the negative gradient of the objective function, that is,

$$d_k = -\nabla f(x_k).$$

Therefore, the update rule is

$$x_{k+1} = x_k - \gamma_k \nabla f(x_k),$$

where $\gamma_k > 0$ in our case is a fixed step size.

Algorithm 1 Steepest Descent Method

Require: Function f , gradient ∇f , initial point x_0 , step size γ , tolerance $\epsilon > 0$

Ensure: Approximate minimizer x^*

```

1: Set  $x_k \leftarrow x_0, k \leftarrow 0$ 
2: while  $\|\nabla f(x_k)\| > \epsilon$  do
3:   Compute gradient  $\nabla f(x_k)$ 
4:    $d_k \leftarrow -\nabla f(x_k)$  ▷ Descent direction
5:    $x_{k+1} \leftarrow x_k + \gamma d_k$ 
6:    $k \leftarrow k + 1$ 
7: end while
8: return  $x^* \leftarrow x_k$ 

```

2.2 Convergence Analysis for $f(x, y)$

Consider our quadratic objective function

$$f(x, y) = \frac{1}{3}x^2 + 3y^2$$

It can be expressed in matrix form as

$$f(x, y) = \frac{1}{2} \begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} 2/3 & 0 \\ 0 & 6 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \frac{1}{2} x^T Q x, \quad \text{where } Q = \begin{bmatrix} 2/3 & 0 \\ 0 & 6 \end{bmatrix}$$

and its gradient is equal to

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{pmatrix} = Qx$$

Substituting the gradient into the update rule of Steepest Descent method gives

$$x_{k+1} = x_k - \gamma \nabla f(x_k) = x_k - \gamma Q x_k = (I - \gamma Q) x_k$$

Applying this iteratively, we obtain

$$x_{k+N} = (I - \gamma Q)^N x_k = \begin{bmatrix} (1 - \frac{2}{3}\gamma)^N & 0 \\ 0 & (1 - 6\gamma)^N \end{bmatrix} x_k$$

The vector x_{k+N} converges to the global minimum $x^* = (0, 0)$ if and only if:

$$\lim_{N \rightarrow \infty} (1 - \frac{2}{3}\gamma)^N = 0 \quad \text{and} \quad \lim_{N \rightarrow \infty} (1 - 6\gamma)^N = 0$$

These conditions lead to the inequalities

$$|1 - \frac{2}{3}\gamma| < 1 \implies -1 < 1 - \frac{2}{3}\gamma < 1 \implies 0 < \gamma < 3$$

$$|1 - 6\gamma| < 1 \implies -1 < 1 - 6\gamma < 1 \implies 0 < \gamma < \frac{1}{3}$$

The more restrictive condition dominates, so the step size must satisfy

$$\boxed{0 < \gamma < \frac{1}{3}}$$

to ensure convergence of the Steepest Descent method for this quadratic function.

2.3 Results

The experimental results from the execution of the Steepest Descent algorithm for different fixed step sizes, starting from $(5, 5)$, summarized in Table 1 and illustrated in Figure 3, perfectly validate the theoretical convergence condition $0 < \gamma < \frac{1}{3}$ derived in the previous section.

- **Convergence (Stable Step Sizes):** For the step sizes $\gamma = 0.1$ and $\gamma = 0.3$, which both fall within the theoretical convergence interval $(0, \frac{1}{3})$, the algorithm successfully converged to points very close to the global minimum $x^* = (0, 0)$, where $f(x^*) = 0$.
- **Effect of Step Size on Speed:** Although both converged, the larger step size of $\gamma = 0.3$ resulted in significantly faster convergence, requiring only 37 iterations, compared to 86 iterations for $\gamma = 0.1$. This is because a larger γ corresponds to a higher learning rate, accelerating convergence.
- **Divergence (Unstable Step Sizes):** On the contrary, when the step size was chosen outside the convergence range ($\gamma = 3.0$ and $\gamma = 5.0$), the algorithm diverged. The iterative points moved exponentially away from the minimum with each step.
- **Numerical Instability:** As shown in the table, for $\gamma = 3.0$ and $\gamma = 5.0$, the algorithm ran for the maximum iterations 50,000 without meeting the tolerance ϵ . The function values grew so large that they quickly resulted in Not a Number (NaN) values due to floating-point overflow.

This comparison confirms that for a fixed step size method, the convergence of the algorithm is highly dependent on the strict bounds imposed by the maximum eigenvalue of the Hessian matrix.

Step Size γ	Minimum Point x^*	Minimum Value $f(x^*)$	Iterations
0.1	(0.013247, 0.0)	0.000058	86
0.3	(0.001298, -0.001298)	0.000006	37
3.0	(5.0, NaN)	NaN	50000
5.0	(NaN, NaN)	NaN	50000

Table 1: Results of the Steepest Descent method for different step sizes starting at (5, 5).

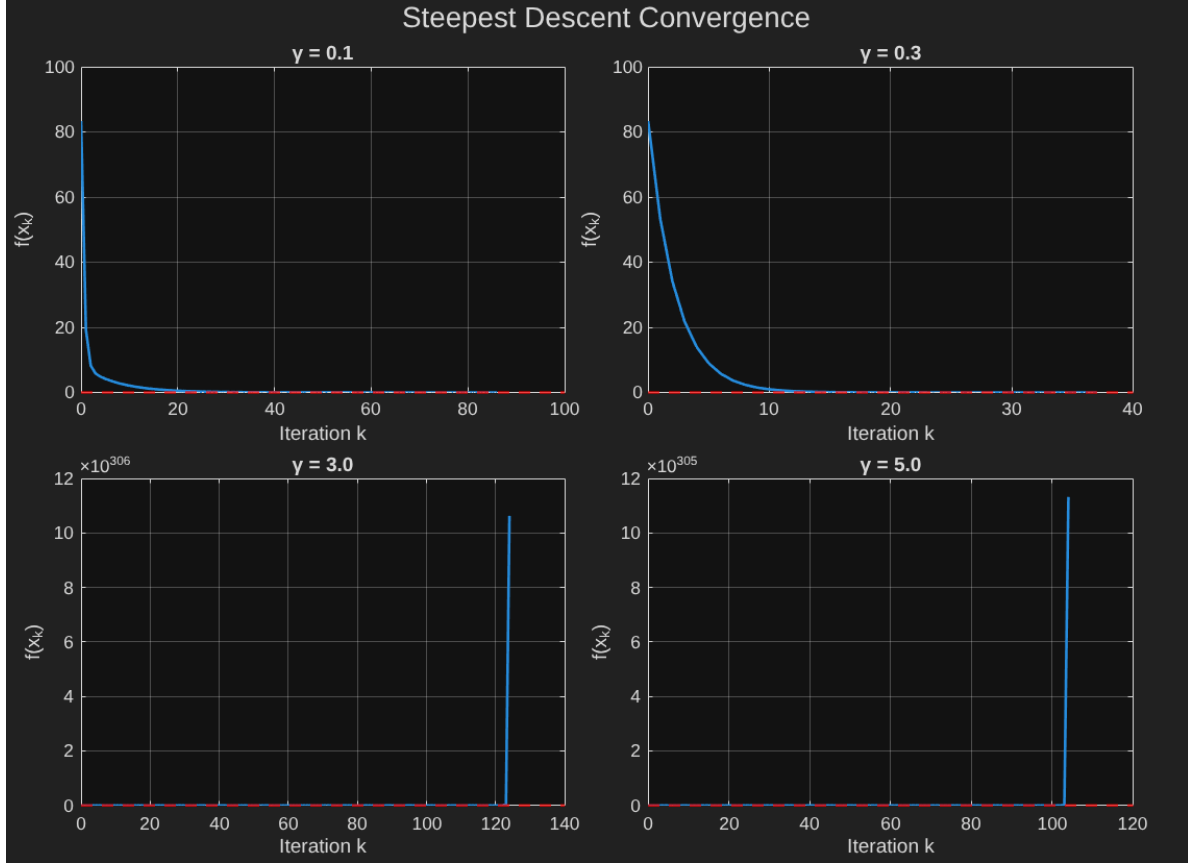


Figure 3: Convergence of the Steepest Descent method for different step sizes starting at (5, 5).

3 Projected Steepest Descent Method

3.1 Method Description

The Projected Steepest Descent method is an extension of the classical Steepest Descent algorithm designed for *constrained* optimization problems. In our setting, the feasible region is the closed and convex rectangle

$$X = \{(x, y) \in \mathbb{R}^2 : -10 \leq x \leq 5, -8 \leq y \leq 12\}.$$

At each iteration, instead of moving directly along the negative gradient, the algorithm first computes an *unconstrained* descent step and then projects this point back onto the feasible set. The procedure consists of the following steps:

- Starting from the current iterate x_k , we compute an intermediate point

$$x'_k = x_k - s\nabla f(x_k),$$

where $s > 0$ is a step size. This point represents the direction of the steepest descent, as in the unconstrained case.

- Since x'_k may lie outside the feasible region, it is projected onto the constraint set X :

$$\bar{x}_k = \text{Pr}_X(x_k - s\nabla f(x_k))$$

For the rectangular set X , the projection is performed componentwise by simply clipping each coordinate into the corresponding interval.

- The vector

$$d_k = \bar{x}_k - x_k$$

defines a feasible descent direction that always keeps the next iterate inside X .

- Finally, the updated iterate is obtained using a fixed step size $\gamma \in (0, 1]$:

$$x_{k+1} = x_k + \gamma d_k.$$

If γ and s are chosen appropriately, the method generates a sequence of feasible points $\{x_k\}$ that converges to a minimizer of f over the set X . This projection-based correction ensures that every iterate respects the box constraints, making the method suitable for constrained optimization problems where the classical Steepest Descent algorithm would otherwise leave the feasible region.

Algorithm 2 Projected Steepest Descent Method

Require: Function f , gradient ∇f , initial point x_0 , step size γ , step size s , tolerance $\epsilon > 0$

Ensure: Approximate minimizer x^*

```

1: Set  $x_k \leftarrow x_0$ ,  $k \leftarrow 0$ 
2: while  $\|\nabla f(x_k)\| > \epsilon$  do
3:   Compute gradient  $\nabla f(x_k)$ 
4:    $\bar{x}_k \leftarrow \text{Pr}_X\{x_k - s\nabla f(x_k)\}$  ▷ Projection of  $x_k$  onto the feasible region  $X$ 
5:    $d_k \leftarrow \bar{x}_k - x_k$  ▷ Descent direction
6:    $x_{k+1} \leftarrow x_k + \gamma d_k$ 
7:    $k \leftarrow k + 1$ 
8: end while
9: return  $x^* \leftarrow x_k$ 

```

3.2 Results

3.2.1 Case 1: $x_0 = (5, -5)$, $\gamma = 0.5$, $s = 5$

In this experiment we apply the Projected Steepest Descent method starting from the initial point

$$x_0 = (5, -5), \quad \gamma = 0.5, \quad s = 5,$$

with the feasible region defined as the rectangle

$$X = \{(x, y) \in \mathbb{R}^2 : -10 \leq x \leq 5, -8 \leq y \leq 12\}.$$

Since the initial point lies strictly inside the feasible set, the first intermediate point is computed as

$$x'_k = x_k - s \nabla f(x_k),$$

and for all early iterations this point remains inside X . Thus, the projection operator has no effect, meaning $\bar{x}_k = x'_k$ and the update rule reduces to

$$x_{k+1} = x_k + \gamma(\bar{x}_k - x_k) = x_k - \gamma s \nabla f(x_k).$$

This is exactly the same update rule as the classical Steepest Descent method with an *effective* step size

$$\gamma_{\text{eff}} = \gamma s.$$

From the convergence analysis in Section 2, the Steepest Descent method converges for this quadratic function only if

$$0 < \gamma_{\text{eff}} < \frac{1}{3}.$$

However, in this experiment we have

$$\gamma_{\text{eff}} = \gamma s = 0.5 \times 5 = 2.5,$$

which lies outside the convergence interval. Therefore, the method cannot converge and the iterates diverge away from the minimizer.

Figure 4 shows the evolution of the objective function value $f(x_k)$ over the iterations. We observe that the method reaches the maximum iteration limit (50,000 iterations) without approaching the true minimum value $f^* = 0$.

Unlike the unconstrained Steepest Descent method with large step sizes (e.g., $\gamma = 3$ or $\gamma = 5$), the projected method keeps all iterates *inside* the feasible region. Thus, although the iterates diverge numerically, the projection operator prevents them from escaping the constraint set.

Improvement by adjusting the parameter s . If we reduce the value of s to $s = 0.5$ while keeping $\gamma = 0.5$, then the effective step size becomes

$$\gamma_{\text{eff}} = \gamma s = 0.5 \times 0.5 = 0.25,$$

which *does* satisfy the convergence condition

$$0 < \gamma_{\text{eff}} < \frac{1}{3}.$$

In this case, the method converges smoothly to the global minimum $(0, 0)$ in only 30 iterations, fully consistent with the theoretical prediction.

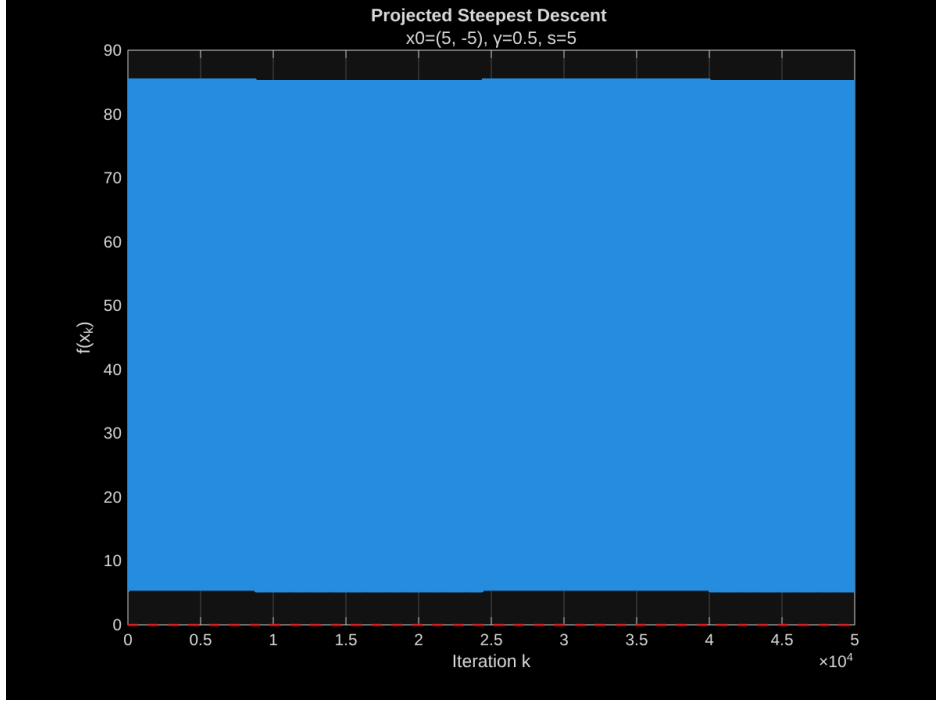


Figure 4: Convergence of the Projected Steepest Descent method for $x_0 = (5, -5)$, $\gamma = 0.5$, $s = 5$.

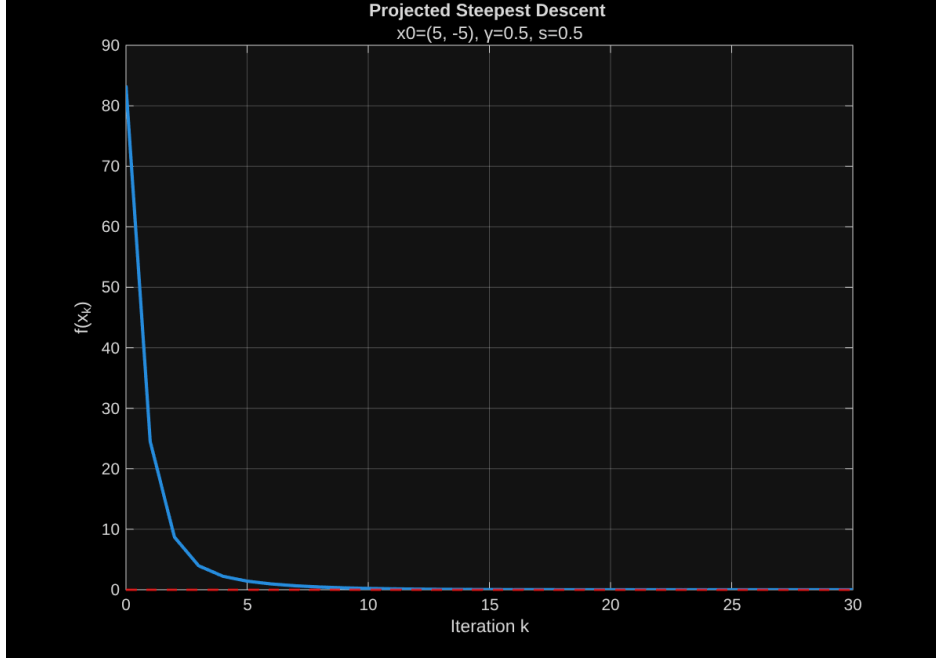


Figure 5: Convergence of the Projected Steepest Descent method for $x_0 = (5, -5)$, $\gamma = 0.5$, $s = 0.5$.

3.2.2 Case 2: $x_0 = (-5, 10)$, $\gamma = 0.1$, $s = 15$

In the second experiment, we apply the Projected Steepest Descent method starting from the point

$$x_0 = (-5, 10), \quad \gamma = 0.1, \quad s = 15.$$

The fixed step size is within the theoretical convergence range derived earlier,

$$0 < \gamma < \frac{1}{3},$$

so we expect the method to converge. Indeed, the algorithm eventually approaches the minimizer of the function; however, its behavior is highly irregular.

Effect of a large internal step size s . Although γ is small, the intermediate update

$$x'_k = x_k - s \nabla f(x_k)$$

uses a very large value of $s = 15$, so the point x'_k undergoes a drastic displacement at each iteration. This leads to several phenomena:

- The point x'_k often falls outside the feasible region X , forcing the projection operator to clip its components.
- The resulting direction $d_k = \bar{x}_k - x_k$ oscillates heavily, causing the objective function values to fluctuate.
- As a result, the method converges very slowly and exhibits *jittering* behavior before stabilizing.

Figure 6 confirms this effect: even though the sequence eventually converges, the objective function decreases in an unstable way. In fact, the method requires 22,419 iterations to reach the tolerance threshold, despite using a theoretically valid step size γ .

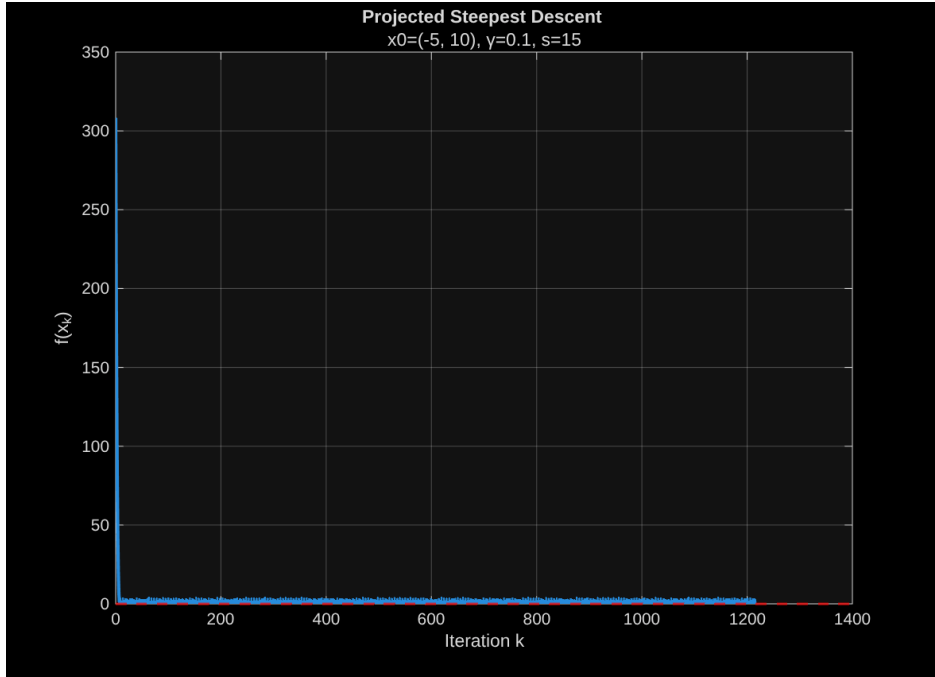


Figure 6: Convergence of the Projected Steepest Descent method for $x_0 = (-5, 10)$, $\gamma = 0.1$, $s = 15$.

Improving the internal step parameter s . To improve the behavior of the method, we reduce the value of s from 15 to 3.

With the smaller value of s , the algorithm converges in only 37 iterations, without oscillations, and reaches the global minimum $(0, 0)$ efficiently. This demonstrates that although γ determines theoretical convergence, the parameter s controls the *practical stability* of the Projected Steepest Descent method.

Figure 7 illustrates the improved convergence behavior.

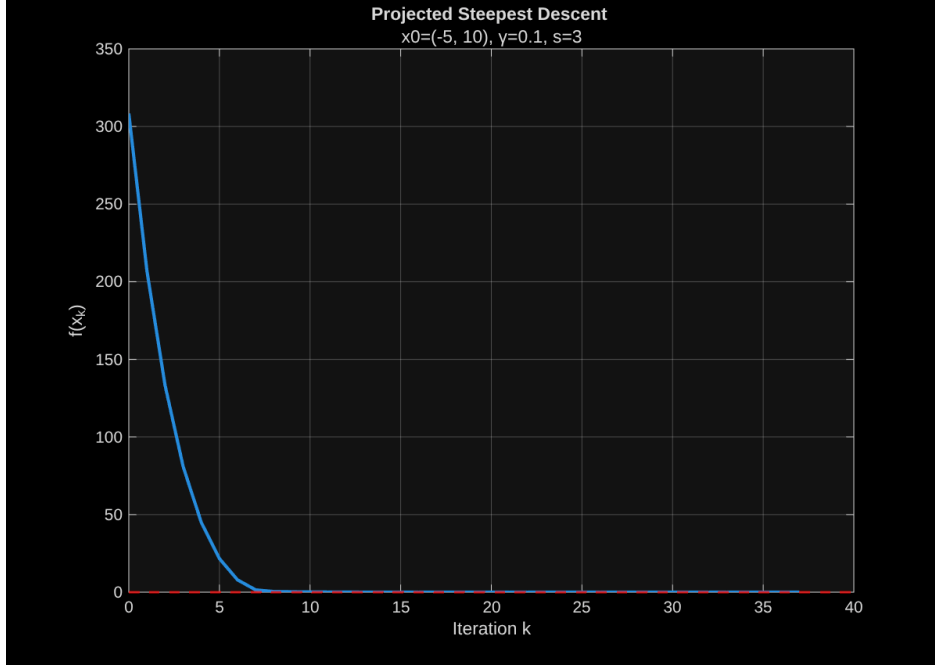


Figure 7: Convergence of the Projected Steepest Descent method for $x_0 = (-5, 10)$ using $s = 3$ and $\gamma = 0.1$.

3.2.3 Case 3: $x_0 = (8, -10)$, $\gamma = 0.2$, $s = 0.1$

In the final experiment, we consider the initial point

$$x_0 = (8, -10), \quad \gamma = 0.2, \quad s = 0.1.$$

Both parameters satisfy the theoretical convergence condition so the algorithm is expected to converge smoothly. However, the starting point lies *outside* the feasible region

$$X = \{(x, y) \in \mathbb{R}^2 : -10 \leq x \leq 5, \quad -8 \leq y \leq 12\}.$$

This does not pose a problem, because the Projected Steepest Descent method ensures feasibility by applying the projection operator:

$$\bar{x}_k = \text{Pr}_X(x_k - s\nabla f(x_k)).$$

The numerical results show that the method successfully converges to the global minimum after 277 iterations. The corresponding convergence plot is shown in Figure 8. As expected, the objective function decreases monotonically without oscillations, reflecting the good choice of parameters.

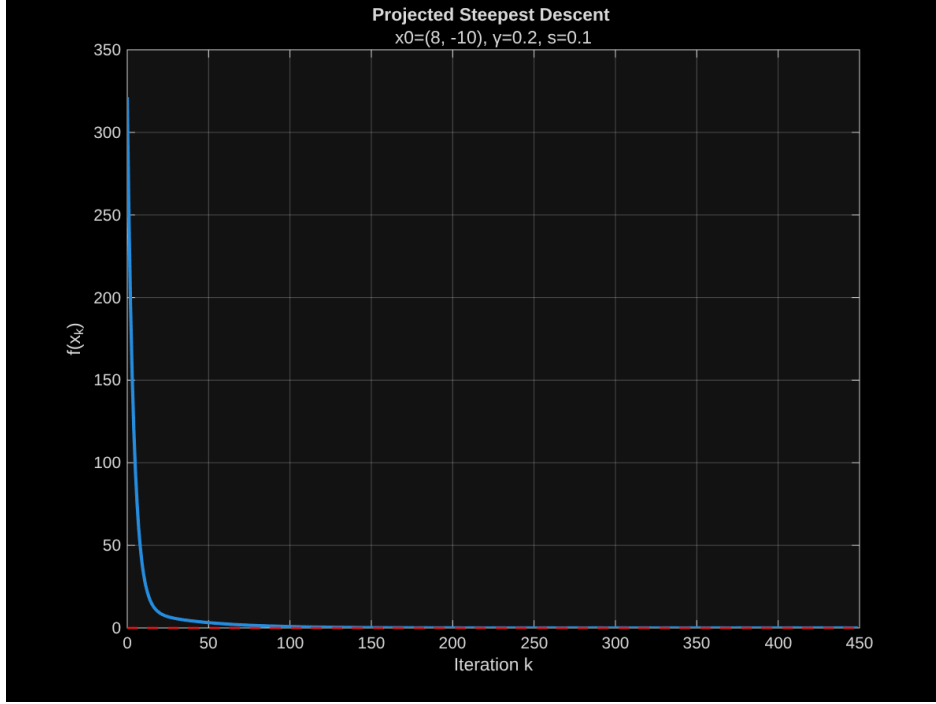


Figure 8: Convergence of the Projected Steepest Descent method for $x_0 = (8, -10)$, $\gamma = 0.2$, $s = 0.1$.

Improving the initialization. One practical improvement is to *project the initial point before starting the iterations*. Instead of beginning with the infeasible $x_0 = (8, -10)$, we may start directly from:

$$x_0^{\text{proj}} = \text{Pr}_X(8, -10) = (5, -8).$$

This reduces the initial distance from the optimum and therefore accelerates convergence. Indeed, when using x_0^{proj} as the starting point, the method converges in 263 iterations instead of 277.

Figure 9 illustrates this faster descent.

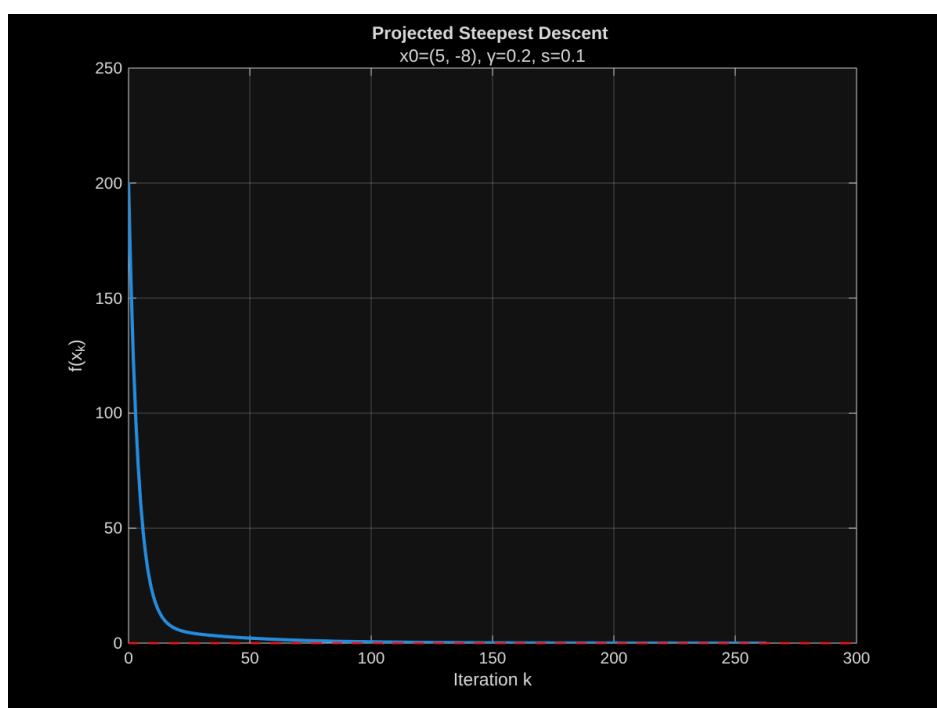


Figure 9: Convergence of the Projected Steepest Descent method for $x_0 = (5, -8)$, $\gamma = 0.2$, $s = 0.1$.